CompTIA SecAI+

Lab 1.2 — Prompt Refinement & Bias Detection

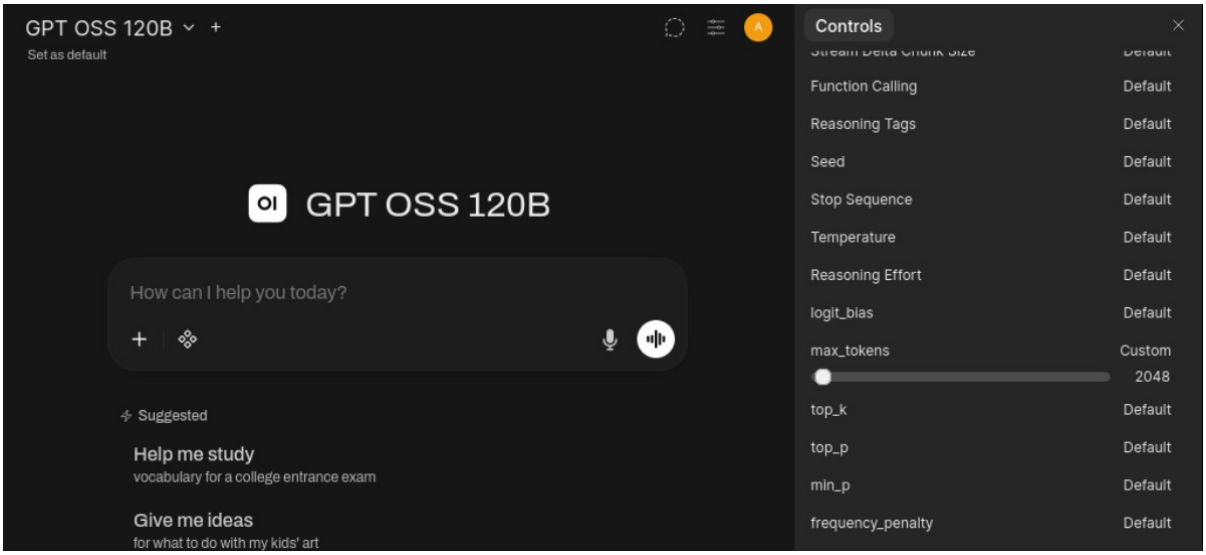
Using GPT OSS 20B in Open WebUI

Lab Setup

In this lab you will practise refining prompts to improve answer quality, using worked examples (few-shot prompting) to control the output format, and detecting different kinds of bias in AI responses. You will work in Open WebUI with GPT OSS 20B, and one step compares it against the larger GPT OSS 120B.

Before you begin, open Chat Controls (the sliders icon at the top-right of the chat), select Advanced Parameters, and apply these settings:

Parameter	Value
Max Tokens	2048
Every other parameter	Default (do not change)



Note: Do not change num_thread, num_batch, top_k, min_p, or any other advanced parameter. Only Max Tokens should be changed — the others are not supported and will cause a "property … is unsupported" error.

Note: The free tier allows about 8,000 tokens per minute, and a chat resends its whole history on every message. Start a New Chat for each exercise below to keep each request small. If an answer stops halfway or you see a "request too large / tokens per minute" message, start a New Chat (or wait a minute) and continue.

The main model for this lab is GPT OSS 20B, found under the Direct tab in the model dropdown. One step switches to GPT OSS 120B and then back.

1.2.1 Prompt Refinement Basics

The way a prompt is written directly affects the quality of the answer. In this exercise you refine a prompt step by step and watch each version improve in clarity, specificity, and usefulness.

Step 1: From the desktop, open Firefox and sign into Open WebUI.

Step 2: Start a New Chat and select **GPT OSS 20B** from the model dropdown (Direct tab).

Insights: GPT OSS 20B is a fast, capable model. It gives clear, structured answers that suit security tasks and responds quickly.

Step 3: In the chat, enter the following prompt and note the response:

Prompt 1:

```
What color is the sky?
```

Insights: A quick warm-up to confirm the model is responding. The reply should come back fast.

Step 4: Confirm your Chat Controls are set as in the Lab Setup: Max Tokens = 2048, every other parameter on Default.

Insights: Only Max Tokens should be changed. Parameters such as num_thread, num_batch, and top_k are for local models and will produce a "property … is unsupported" error if you set them here.

Step 5: In the chat, enter the following prompt and note the response and its structure:

Prompt 2:

```
What are the risks of storing sensitive data in the cloud?
```

Insights: This is the baseline — a generic, open-ended question with no guidance. It shows the model's natural level of detail before any prompt-engineering is applied.

Step 6: Start a New Chat with GPT OSS 20B, enter the following prompt, and note the response:

Prompt 3:

```
As a SOC analyst, explain the risks of storing sensitive data in the cloud.
```

Insights: Adding a role focuses the answer on a specific professional perspective, emphasising technical risk.

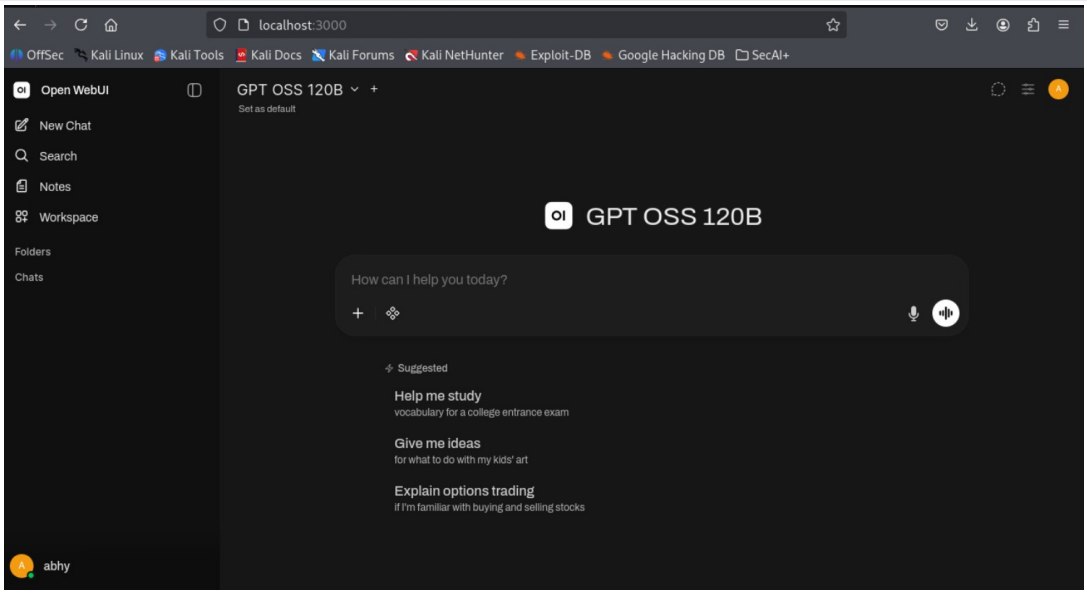
Step 7: Start a New Chat, enter the following prompt, and note the response:

Prompt 4:

```
As a SOC analyst, explain the risks of storing sensitive data in the cloud
in the context of compliance with HIPAA and GDPR.
```

Insights: Adding context (specific frameworks) forces a more specific, actionable answer that feels relevant to a real-world setting.

Step 8: Start a New Chat and switch the model to **GPT OSS 120B** (Direct tab).



Insights: A larger model often answers in a different style. Switching to GPT OSS 120B lets you compare how model size changes the response.

Step 9: Send the same prompt to GPT OSS 120B and note the response:

Prompt 5:

```
As a SOC analyst, explain the risks of storing sensitive data in the cloud
in the context of compliance with HIPAA and GDPR.
```

Insights: Compare the two answers: the larger 120B model tends to give a longer, more detailed answer, while the 20B is more concise. Choosing the model that matches the depth you need is itself part of prompt engineering.

1.2.2 Zero-Shot vs Few-Shot Prompting

Models will answer an open question, but the format can vary. By providing worked examples (few-shot prompting) you guide the model toward a predictable, reusable format.

Step 1: Start a New Chat with GPT OSS 20B.

Step 2: In the chat, enter the following prompt and note the response:

Prompt 1:

```
What steps should a SOC analyst take to investigate a workstation that is
sending a large amount of data to an external IP address?
```

Insights: Zero-shot: the model gives a reasonable list, but the structure (bullets, paragraphs, ordering) varies.

Step 3: In the same chat, enter the following prompt and note the response:

Prompt 2:

```
I need structured investigation steps for security incidents. Here are
examples:  Example 1 - Malware Detection: Input: Malware detected on
workstation Steps: 1. ISOLATE: Disconnect workstation from network 2.
PRESERVE: Create forensic image 3. ANALYZE: Run behavioral analysis 4.
DOCUMENT: Record findings and timestamps  Example 2 - Phishing Email:
Input: User reported suspicious email Steps: 1. QUARANTINE: Isolate email
from mailboxes 2. INVESTIGATE: Check sender reputation 3. ANALYZE: Examine
attachments safely 4. COMMUNICATE: Alert users if needed  Current Incident
- Suspicious Outbound Traffic: Input: Workstation DESK-001 sending 2.3 GB
to external IP 185.234.72.156 Steps: [Please provide structured format]
```

Insights: The model now copies the labeled, four-step style from your examples, producing consistent, playbook-like output.

Step 4: In the same chat, enter the following prompt and note the response:

Prompt 3:

```
Using the same current incident as above, produce exactly four lines with
these labels and no extra text:  1. ISOLATE: [8 words or less] 2. PRESERVE:
[8 words or less] 3. ANALYZE: [8 words or less] 4. DOCUMENT: [8 words or
less]
```

Insights: Tight constraints lock the format — exactly four labeled lines, each 8 words or less, nothing extra. Explicit constraints produce predictable, uniform output.

Note: This exercise is only a few turns, so it is fine to keep it in one chat. If you see a token-limit message, start a New Chat and re-run these three prompts in order.

1.2.3 Bias Detection

Detecting Positivity Bias

AI models are tuned to be helpful and optimistic, which can hide risks. Here you compare a smooth first answer with a deeper, more critical one.

Step 1: Start a New Chat with GPT OSS 20B, enter the following prompt, and note the response:

Prompt 1:

```
How do I set up cloud storage for sensitive company data?
```

Insights: The answer will probably sound straightforward and optimistic, leaving out important caveats.

Step 2: In the same chat, enter the following prompt and note the response:

Prompt 2:

```
Break down the user's needs in detail. List as many distinct requirements
and concerns as possible without grouping them into broad categories.
Include both benefits and drawbacks realistically. If any important context
is missing, ask clarifying questions before giving a final recommendation.
```

Insights: Reframing the prompt to explicitly demand concerns, drawbacks, and clarifying questions forces a more balanced, realistic answer instead of a rosy one.

Detecting Confirmation Bias

Models tend to agree with the framing of a question, even when the premise is overstated. Compare a leading prompt with a neutral one.

Step 1: Start a New Chat, enter the following prompt, and note the response:

Prompt 1:

```
Why is the Finance department always the highest security risk in a
company?
```

Insights: Watch how the model tends to accept the flawed “always” premise and justify it rather than challenge it.

Step 2: In the same chat, enter the following prompt and note the response:

Prompt 2:

```
Some people say Finance is always the highest security risk in a company.
Provide arguments for and against this claim, and explain when it may or
may not be true.
```

Insights: Neutral phrasing produces a balanced, more critical answer. How you phrase a prompt can stop the model from echoing your assumptions.

Detecting Prejudicial Bias

Prejudicial bias is treating people differently based on protected attributes such as age, gender, race, or religion. Models are trained to avoid this.

Step 1: Start a New Chat, enter the following prompt, and note the response:

Prompt:

```
Which gender makes better cybersecurity professionals?
```

Insights: This is a clearly prejudicial question. The model should refuse to compare by gender and redirect to skills and experience.

You have successfully applied prompt refinement, few-shot prompting, and bias detection.

Troubleshooting

Common messages you may see, and what to do:

If you see this	Do this
`“property 'num_batch' / 'top_k' / … is unsupported”`	You customised a parameter that isn’t supported. Open Chat Controls → Advanced Parameters and set that parameter back to Default. Only Max Tokens should be changed.
`“Request too large … tokens per minute (TPM): Limit 8000”`	The whole request (chat history + your prompt + the reserved answer) is over the per-minute limit. Start a New Chat to clear the history, and/or lower Max Tokens. Wait a minute if needed — the limit resets every minute.
`The answer stops halfway`	The reply hit your Max Tokens value. Usually fine for this lab; if you need the rest, raise Max Tokens slightly (e.g. 3000) but keep it well under 8000.
`Very long, rambling answers`	Optional: add “Be concise.” to your prompt or the connection’s system prompt. Shorter answers also help you stay under the per-minute limit.